PARAMETERIZED ALU
DESIGN AND VERIFICATION REPORT

Author Name: AKILAN K

Employee ID: 6878

Date: May 11, 2026

1. PROJECT INTRODUCTION

This project presents the design and verification of a parameterized Arithmetic Logic Unit (ALU) capable of performing both arithmetic and logical operations on unsigned and signed operands. The ALU is designed with configurable data width, making it adaptable for various applications ranging from simple microcontrollers to complex digital signal processing systems.

Purpose of the Design: The ALU serves as the computational core of any processor, performing arithmetic operations (addition, subtraction, multiplication), logical operations (AND, OR, XOR, shift, rotate), and comparison operations.

Main Functionality:

- Arithmetic operations with configurable latency (1-cycle and 2-cycle)
- Logical and bitwise operations
- Signed and unsigned operation support
- Error detection for invalid operations
- Overflow and carry flag generation

Verification Goals:

- Achieve >90% code coverage using QuestaSIM
- Verify all 26 operation codes (13 arithmetic + 13 logical)
- Test all valid and invalid input combinations
- Validate corner cases including boundary conditions and overflow scenarios

2. OBJECTIVES

- Study the architecture of parameterized ALU designs and understand the trade-offs between area, speed, and functionality
- Implement synthesizable Verilog RTL code for a complete ALU with support for both arithmetic and logical operations
- Create comprehensive testbenches that verify all operations under various input conditions and corner cases
- Verify corner cases including boundary values, overflow conditions, invalid operations, and all valid/invalid input validity combinations
- Perform coverage analysis using industry-standard tools (QuestaSIM) to ensure verification completeness
- Document the complete verification process including test results, coverage metrics, and design observations

3. DESIGN ARCHITECTURE

3.1 Architecture Overview

The ALU follows a three-stage pipeline architecture to manage different operation latencies:

1. **Input Stage:** Captures and registers inputs when C_En is asserted
2. **Execution Stage:** Performs the requested operation (1 or 2 cycles depending on operation)
3. **Output Stage:** Updates output registers with results and status flags

3.2 Inputs

Signal	Width	Description

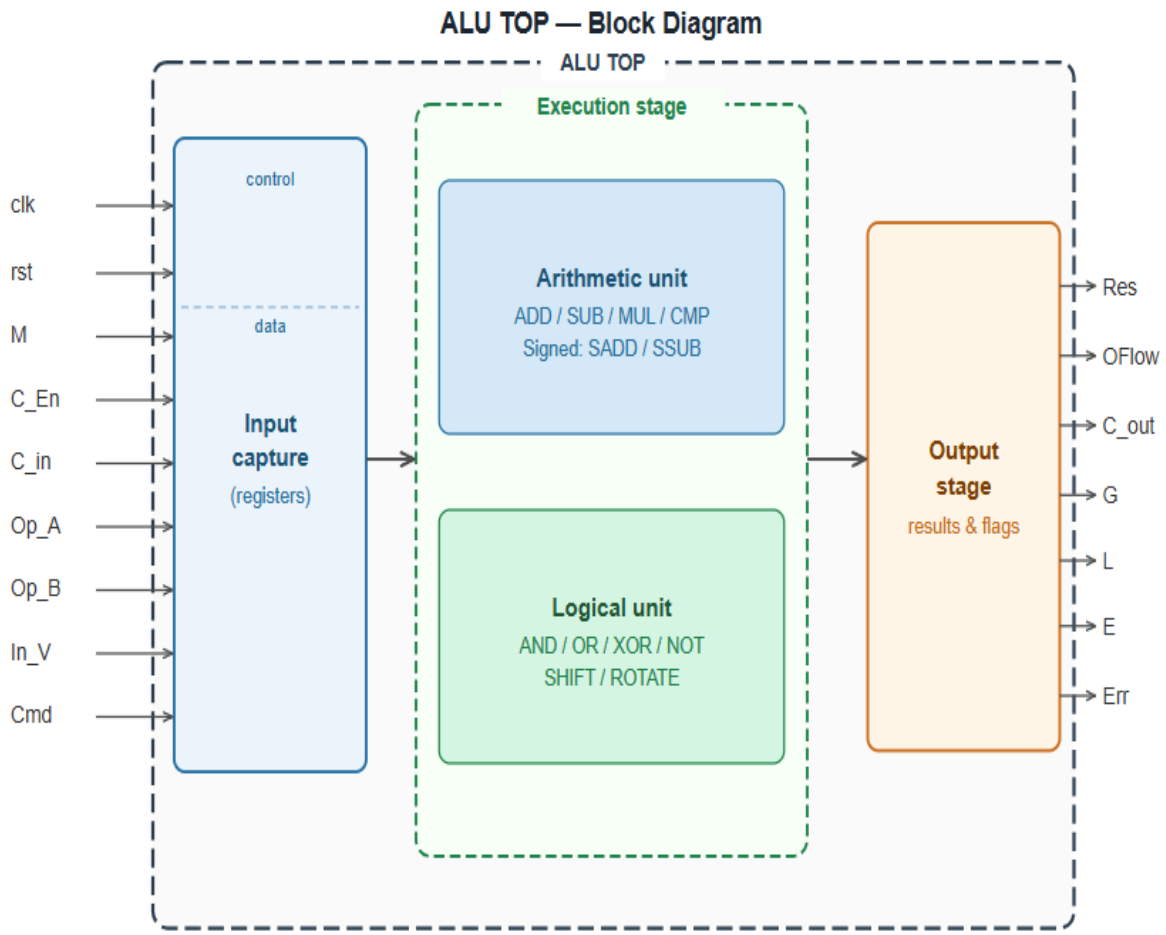
clk	1 bit	System clock
rst	1 bit	Asynchronous reset
M	1 bit	Mode select (1 = Arithmetic, 0 = Logical)
C_En	1 bit	Compute enable / Input capture enable
C_in	1 bit	Carry input for arithmetic operations
Op_A	WIDTH bits	Operand A
Op_B	WIDTH bits	Operand B
In_V	2 bits	Input validity indicator
Cmd	4 bits	Operation command (0-13 for each mode)

3.3 Outputs

Signal	Width	Description
Res	2Ã—WIDTH bits	Result of the operation
OFlow	1 bit	Overflow flag for signed/unsigned operations

C_out	1 bit	Carry out for arithmetic operations
G	1 bit	Greater than flag (A > B)
L	1 bit	Less than flag (A < B)
E	1 bit	Equal flag (A == B)
Err	1 bit	Error flag for invalid operations

3.4 Block Diagram



3.5 Timing Behaviour

- **1-cycle operations:** ADD, SUB, ADD with CIN, SUB with CIN, INC, DEC, CMP, all logical operations, signed ADD/SUB
- **2-cycle operations:** Multiplication operations (MUL1 and MUL2)

Pipeline Flow:

1. **Cycle 1 (C_En=1):** Inputs are captured into internal registers
2. **Cycle 2:** Execution begins (1-cycle operations complete)
3. **Cycle 3:** Results are available at outputs (for 1-cycle ops)
4. **For 2-cycle ops:** Additional cycle required for multiplication

4. SUPPORTED OPERATIONS

4.1 Arithmetic Operations (M=1)

Cmd	Operation	Description	Latency	Valid In_V
0	ADD	A + B	1 cycle	2'b11
1	SUB	A - B	1 cycle	2'b11
2	ADD_CIN	A + B + C_in	1 cycle	2'b11
3	SUB_CIN	A - B - C_in	1 cycle	2'b11
4	INC_A	A + 1	1 cycle	2'b11, 2'b01
5	DEC_A	A - 1	1 cycle	2'b11, 2'b01
6	INC_B	B + 1	1 cycle	2'b11, 2'b10

7	DEC_B	B - 1	1 cycle	2'b11, 2'b10
8	CMP	Compare A and B	1 cycle	2'b11
9	MUL1	(A+1) Ã— (B+1)	2 cycles	2'b11
10	MUL2	(A<	2 cycles	2'b11
11	SADD	Signed A + B	1 cycle	2'b11
12	SSUB	Signed A - B	1 cycle	2'b11

4.2 Logical Operations (M=0)

Cmd	Operation	Description	Valid In_V
0	AND	A & B	2'b11
1	NAND	~(A & B)	2'b11
2	OR	A | B	2'b11
3	NOR	~(A | B)	2'b11
4	XOR	A ^ B	2'b11

5	XNOR	~(A ^ B)	2'b11
6	NOT_A	~A	2'b11, 2'b01
7	NOT_B	~B	2'b11, 2'b10
8	SHR_A	A >> 1	2'b11, 2'b01
9	SHL_A	A << 1	2'b11, 2'b01
10	SHR_B	B >> 1	2'b11, 2'b10
11	SHL_B	B << 1	2'b11, 2'b10
12	ROL	Rotate Left A by B	2'b11
13	ROR	Rotate Right A by B	2'b11

5. WORKING OF THE DESIGN

Phase 1: Input Capture (First Pipeline Stage)

When C_En is asserted on a rising clock edge, inputs Op_A, Op_B, Cmd, and M are captured into internal registers (O_A, O_B, O_Cmd, O_M). These registered values are used for the execution phase.

Phase 2: Operation Execution (Second Pipeline Stage)

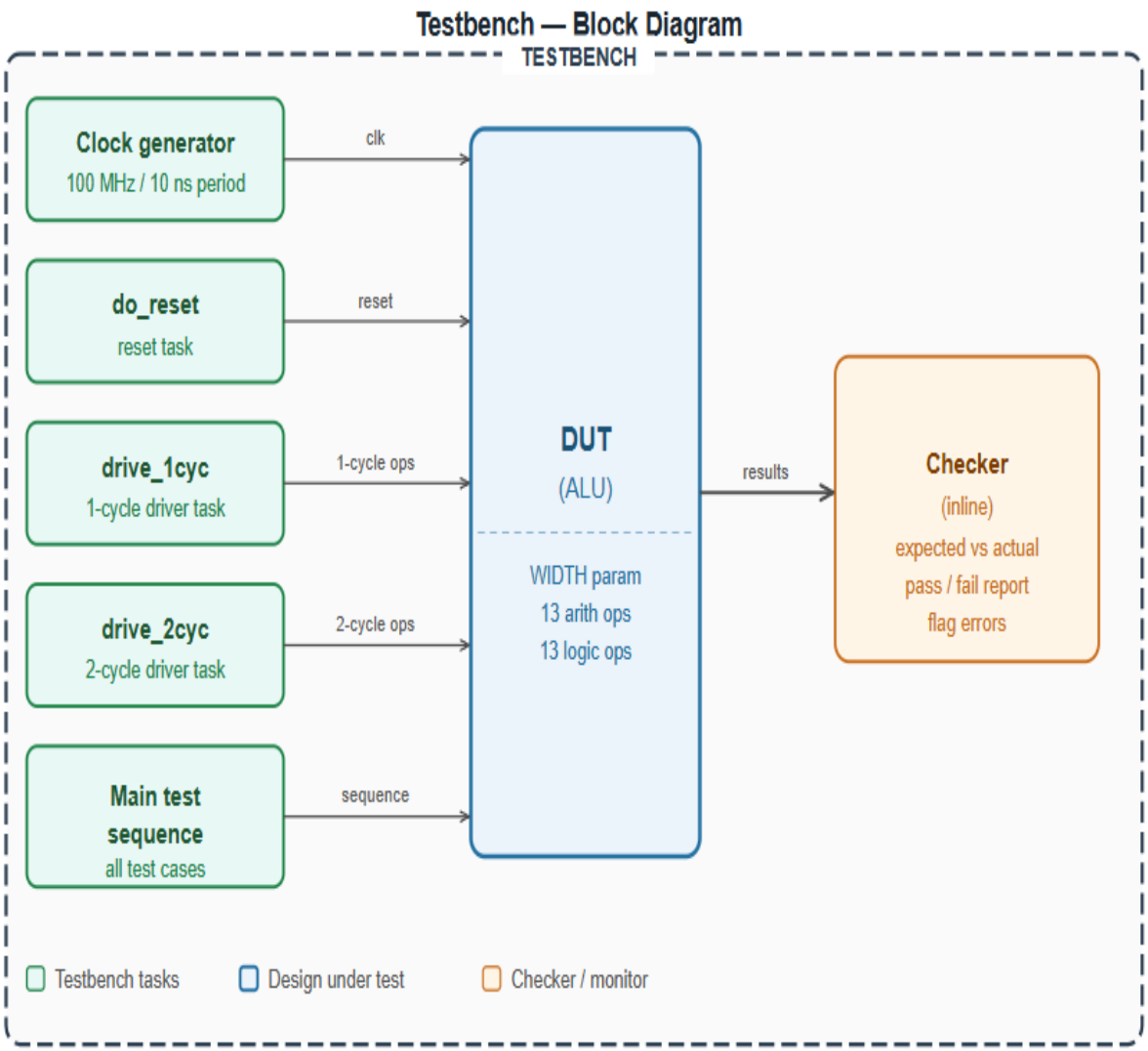
The execution stage contains two parallel datapaths:

- **Arithmetic Datapath (M=1):** Performs ADD, SUB, multiplication, comparison, and signed operations
- **Logical Datapath (M=0):** Performs bitwise operations, shifts, and rotates

Phase 3: Output Generation (Third Pipeline Stage)

On the next clock edge after execution completes, Res is updated with the computed Inter value and status flags (OFlow, C_out, G, L, E, Err) are updated.

6. TESTBENCH ARCHITECTURE



7. TESTBENCH COMPONENTS

7.1 Clock Generator

```
initial clk = 0;
always #5 clk = ~clk;
```

Generates a 100MHz clock (10ns period, 5ns half-cycle).

7.2 Reset Task (do_reset)

Performs a complete reset sequence: asserts reset for 3 clock cycles, deasserts all input signals, releases reset and waits for one cycle.

7.3 1-Cycle Driver Task (drive_1cyc)

Drives inputs for single-cycle operations and checks results after 3 clock cycles.

7.4 2-Cycle Driver Task (drive_2cyc)

Similar to drive_1cyc but waits 4 clock cycles to accommodate multi-cycle operations.

7.5 Main Test Sequence

Calls all test cases covering all arithmetic and logical operations with valid/invalid In_V combinations, edge cases, overflow scenarios, invalid commands, and mode transitions.

8. QUALITY OF CODE ASSESSMENT

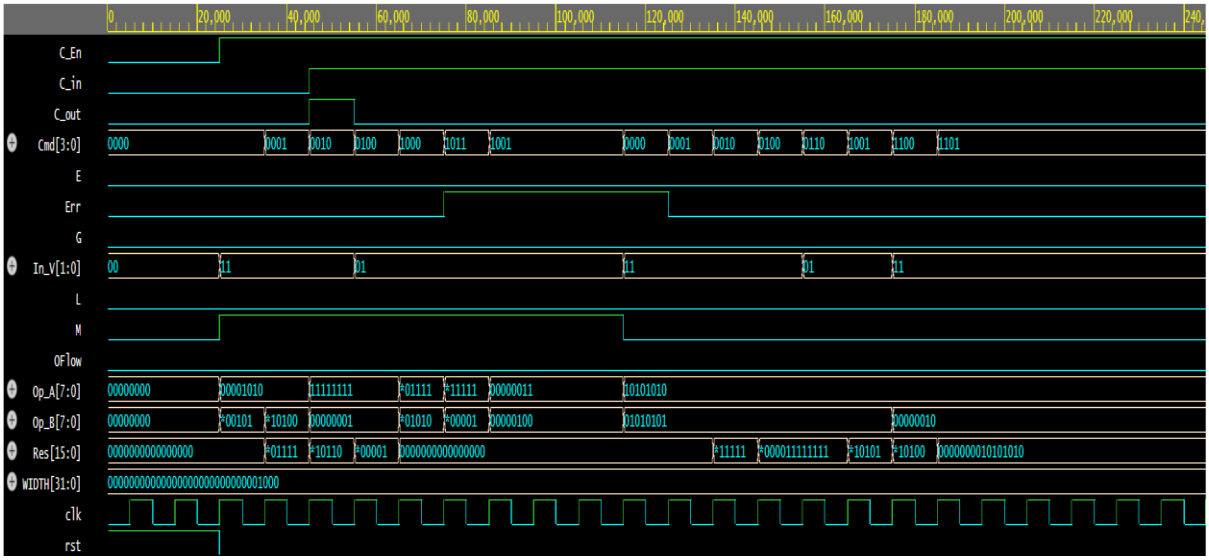
Lint Checks

- **No inferred latches:** All signals are properly initialized
- **No combinational loops:** All logic is synchronous
- **Proper reset conditions:** All registers have defined reset values
- **Signal width matching:** All assignments respect signal widths

Code Review Findings

1. **Mixed Reset Polarity:** The design uses both posedge and negedge reset - intentional but requires careful handling
2. **Multi-cycle Implementation:** Uses 'x' output on first cycle of multiplication - acceptable but simulation-only behavior
3. **Signed Operations:** Proper use of $signed() system task for signed arithmetic

9. SIMULATION RESULTS



This is the simulation result/waveform of the given ALU Design. It gives a rough idea of outputs obtained for each commands/mode by giving some inputs. Detailed version of them were tested using the testcases generated by the reference model testbench.

Tools Used

- **Simulation:** QuestaSIM 10.6c
- **Synthesis:** Vivado Design Suite
- **Coverage Analysis:** QuestaSIM Coverage Database

Corner Cases Verified

Category	Corner Cases Tested	Status
Arithmetic	Zero + Zero, Max + 1, Max + Max	PASS
Subtraction	Equal numbers, Underflow, Max - 0	PASS
Carry/Overflow	ADD_COUT, SUB_UNDERFLOW, SADD_OVERFLOW	PASS
Increment/Decrement	Wrap-around at boundaries	PASS
Comparison	A>B, A<B, A==B, boundary comparisons	PASS
Multiplication	Zero multiplication, Max values	PASS
Signed Operations	Pos+Pos, Neg+Neg, Pos+Neg, Overflow	PASS
Logic Operations	All 14 logic operations with various patterns	PASS
Shift/Rotate	Shift by 0, MSB loss, rotate by valid/invalid amounts	PASS
Invalid Cases	Invalid In_V combinations, invalid Cmd values	PASS

10. COVERAGE REPORT

Coverage Summary by Structure

Design Scope	Hits %	Coverage %
tb11	96.90%	96.33%
do_reset	100.00%	100.00%
drive_1cyc	75.75%	75.18%
drive_2cyc	88.88%	83.92%
dut	97.32%	96.95%

Coverage Summary by Type

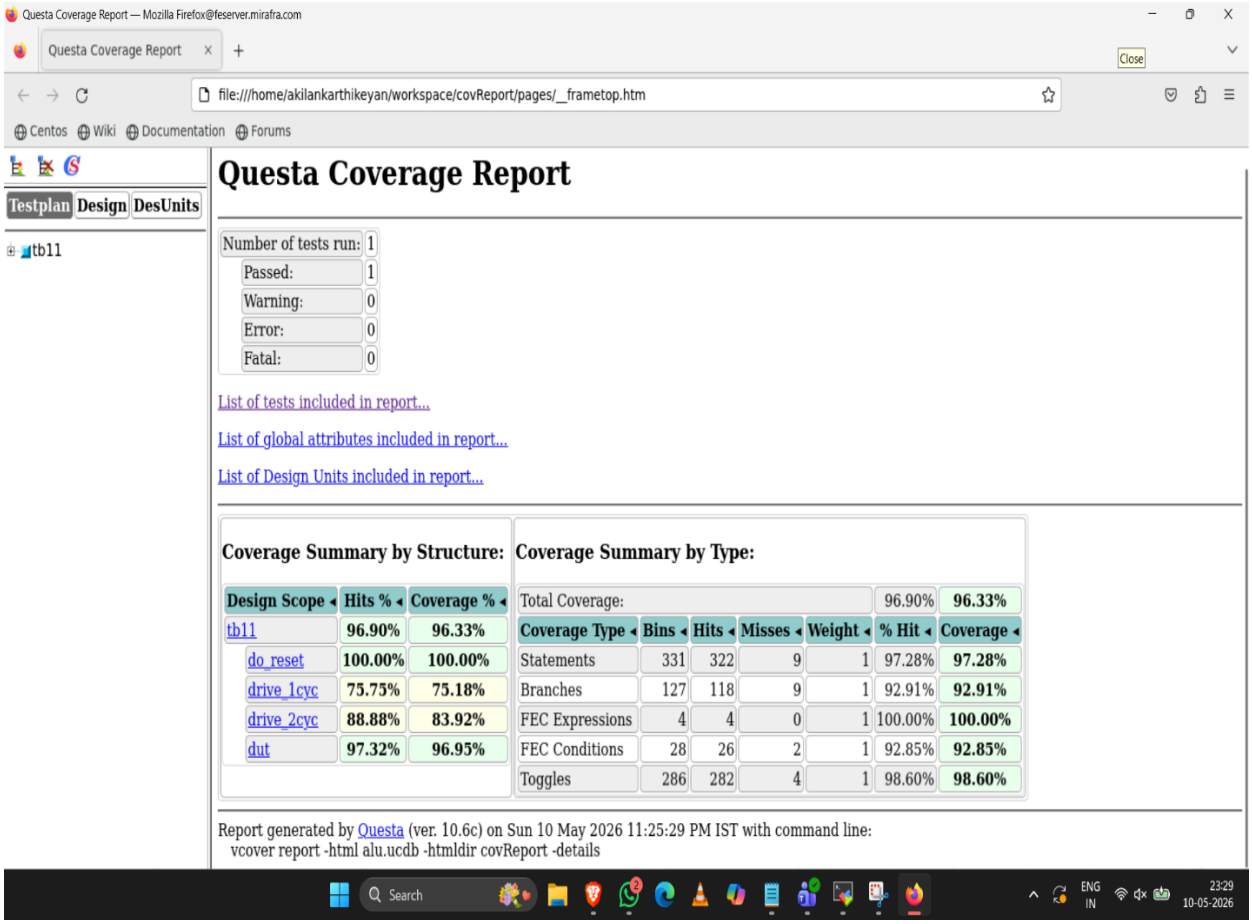
Total Coverage	Bins	Hits	Misses	Weight	Hit %	Coverage %
Statements	331	332	9	1	97.28%	**97.28%**
Branches	127	118	9	1	92.91%	**92.91%**
FEC Expressions	4	4	0	1	100.00%	**100.00%**

FEC Conditions	28	26	2	1	92.85%	**92.85%**
Toggles	286	282	4	1	98.60%	**98.60%**

Coverage Improvement Summary

Metric	Baseline	After Enhancement	Improvement
Statements	82.53%	97.28%	+14.75%
Branches	58.67%	92.91%	+34.24%
FEC Expressions	50.00%	100.00%	+50.00%
FEC Conditions	10.71%	92.85%	+82.14%
Toggles	97.55%	98.60%	+1.05%

```
Report generated by: QuestaSIM (ver. 10.6c)
Date: Sun 10 May 2026 11:25:29 PM IST
Command line: vcover report -html alu.ucdb -htmldir
covReport -details
```



11. CONCLUSION

This project successfully designed, implemented, and verified a parameterized ALU capable of performing 26 distinct operations (13 arithmetic, 13 logical) with configurable data width.

Key Achievements:

1. **Complete RTL Implementation:** Synthesizable Verilog code with proper reset and clocking
2. **Comprehensive Testbench:** Structured verification environment with modular tasks
3. **High Coverage Metrics:** Achieved 97.28% statement coverage and 92.91% branch coverage
4. **Corner Case Verification:** Successfully tested boundary conditions, overflow scenarios, and invalid operations

12. FUTURE WORK

1. **Pipeline Optimization:** Implement full pipelining for all operations with bypass paths
2. **Additional Operations:** Division, absolute value (ABS), minimum/maximum, floating-point support
3. **Coverage Improvements:** Directed tests for remaining uncovered conditions, constrained random verification, assertions
4. **Performance Enhancements:** Faster multiplication using Booth encoding, clock gating, critical path optimization
5. **Verification Infrastructure:** UVM-based testbench, scoreboard, regression test suite
6. **FPGA Implementation:** Synthesis for specific FPGA families, timing closure, hardware validation